Chaîne de caractères en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Les fondamentaux **4**

A. Introduction 4

B. Opérations de base 4

C. Accès et extraction de sous-chaînes 7

D. Exercice 9

E. Exercice : Quiz 9

III - Manipulation des chaînes **11**

A. Introduction 11

B. Transformation 11

C. Recherche et remplacement 13

D. Séparation et jointure 14

E. Exercice : Quiz 17

IV - Techniques avancées **20**

A. Introduction 20

B. Formatage 20

C. Expression régulières 21

D. Fonction et méthodes utiles 22

E. Exercice : Quiz 23

V - Essentiel **27**

Solutions des exercices **28**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 60 minutes

Contexte

Lorsque vous envoyez un message texte ou un courriel, le contenu est une chaîne de caractères. Il en va de même pour un achat en ligne ou une recherche via un moteur de recherche.

Toute application informatisée nécessite des chaînes de caractères pour stocker puis exploiter de l'information. En Python, ces chaînes de caractères s'appellent des strings et sont immuables, ce qui signifie que leur contenu ne peut pas être modifié après leur création. Cette immuabilité permet d'éviter les erreurs liées aux modifications accidentelles, et Python peut ainsi optimiser certaines opérations.

La maîtrise des chaînes de caractères est primordiale pour effectuer un traitement de l'information. À ce titre, nous allons voir comment Python les utilise et les manipule.

Tous droits réservés à STUDI - Reproduction interdite

II Les fondamentaux

A. Introduction

Rappel

Les chaînes de caractères, ou strings, sont l'un des types de données les plus fondamentaux en Python. Elles sont utilisées pour représenter du texte et sont omniprésentes dans la programmation quotidienne. Ensemble, nous allons comprendre comment utiliser les chaînes de caractères et les exploiter.

Cette partie du cours vous introduira aux opérations de base sur les chaînes de caractères ainsi qu'à l'exploitation d'une ou plusieurs chaînes.

B. Opérations de base

Méthode

Python utilise un typage dynamique, ce qui signifie que nous ne sommes pas obligés de préciser le type de la variable. De plus, nous devons utiliser le snake_case dans le nom de nos variables. Il est important de noter qu'il est recommandé de spécifier le type de variable, car cela évite à Python de 'deviner' le type, augmentant ainsi les performances. Un autre avantage de cette spécification est de favoriser la maintenabilité du code. Ainsi, les développeurs qui reprennent votre code sauront directement ce que contient la variable et seront plus efficaces, évitant ainsi le reverse engineering.

Dans un premier temps, nous pouvons déclarer deux variables contenant des chaînes de caractères avec un typage dynamique et static.

```
1 #avec un typage dynamique, python converti "message_stockee" en str
2 Message_stockee = " Bonjour "
3 #avec un typage static, python comprend que "message_static » est une str
4 Message_static : str = "le monde"
```

Ensuite, nous pouvons vérifier le type de « *message_stockee* » via la méthode « *type()* »qui va nous permettre de connaitre si message_stockee est bien un str.

```
1 # avec un typage dynamique, python converti "message_stockee" en str
2 message_stockee = "Bonjour"
3 # cela va afficher: class str
4 print(type(message_stockee))
5 # aves un typage static, python comprend que "message_static" est une str
6 message_static: str = "le monde"
```

Jusqu'à maintenant, nous avons utilisé des doubles guillemets, mais nous pouvons également utiliser des simples guillemets ou encore les deux en même temps !

```
1 # avec un typage dynamique, python senverti "message_stockee" en str
2 message stockee = 'Bonjour'
3 # avec un typage static, python comprend que "message_static" est une str
4 message_static: str = "'Le monde'"
5 print(message_static) # cela va afficher : 'Le monde'
```


Tous droits réservés à STUDI - Reproduction interdite

Attention

Python, étant très souple, permet une utilisation adaptée à vos préférences. Il n'y a pas de recommandation spécifique sur l'utilisation des simples ou des doubles guillemets ; chaque développeur est libre d'utiliser ce qui lui convient.

Méthode

Dans certains cas, nous pouvons avoir besoin de stocker une chaîne de caractères sur plusieurs lignes. Cela permet de maintenir la lisibilité et la maintenabilité du code. Pour cela, il faut utiliser trois doubles guillemets. Cette méthode est très intéressante, car elle permet d'avoir des retours à la ligne sans utiliser le caractère `\n`.

```
1 poeme: str = """Ce texte est un poème, il est écrit sur plusieurs lignes,
2 pour garder sa forme d'origine. """
3 print(poeme)
```

Enfin, en Python, certains caractères que l'on peut mettre dans une chaine peuvent effectuer un saut de ligne, ou encore une tabulation.

- `\t` : permet d'inclure une tabulation (écriture de 3 espaces) ;
- `\n` : effectue un saut de ligne ;
- `\\` : permet de protéger un caractère spécial.

Voici un exemple d'utilisation des deux premiers éléments.

```
1 multiline_message: str = "Première ligne\nDeuxième ligne\nTroisième ligne"
  tabulated_message: str = "Colonne1\tColonne2\tColonne3"
2 print(multiline_message)
3 print(tabulated_message)
```

L'affichage du rendu, donnera cela :

```
Première ligne
Deuxième ligne
Troisième ligne
Colonne1    Colonne2    Colonne3
```

Figure 6 : affichage du rendu

Cependant, il nous reste encore un cas spécial à examiner : le caractère d'échappement. Son utilisation est assez fréquente et extrêmement pratique ! Si, par exemple, nous souhaitons stocker un chemin d'accès dans une variable Python.

```
1 path: str = "C:\Users\NomUtilisateur\Documents"
2 print(path)
```

Le problème, c'est que ce code en apparence très séduisant ne fonctionne pas. Pour preuve, si on essaye de l'exécuter, nous aurons un message d'erreur.

```
    path: str = "C:\Users\NomUtilisateur\Documents"
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXXX escape
```

Tous droits réservés à STUDI - Reproduction interdite

Figure 8 : erreur Python

Python nous dit juste qu'il souhaite un caractère d'échappement sur les « \ », cela donnera:

```
path: str = "C:\\Users\\Nomutilisateur\\Documents" print(path)
```

Le premier « \ » permet de protéger le second « \ » qui sera affiché. Cette protection est obligatoire aussi bien en Python que dans d'autres langages de programmation.

Pour preuve, le print de cette ligne de code, nous donne le rendu escompté :

```
C:\Users\NomUtilisateur\Documents
```

Figure 10 : affichage du rendu de la figure 9

Exemple

Pour rendre cette partie du cours plus concrète, nous allons aider José, un apprenant qui souhaite concevoir un programme en Python pour gérer une petite liste de tâches. Il veut pouvoir ajouter des tâches, visualiser combien il en possède et les exploiter.

```
tache: str = 'Faire les courses'
message: str = 'Tâche :'
```

José, souhaite stocker dans une chaine de caractères « *Tâche : Faire les courses* ». Nous allons devoir utiliser la concaténation, cela permet de « *fusionner* » deux chaines de caractères en une seule via l'opérateur « *+* »

```
tache: str = 'Faire les courses'
message: str = 'Tâche :'
# tache_message contiendra 'Tâche :Faire les courses'
tache_message: str = message + tache
```

À cela, nous allons ajouter un peu d'esthétique en proposant un séparateur après chaque tâche afin de pouvoir en placer une autre. Le but sera de mettre un séparateur de la taille de tache_message.

Par exemple, si tache_message fait une longueur de 20 caractères, nous allons ajouter 20 caractères '-'.

L'approche la plus simple serait de se dire : « *je vais compter les caractères de tache_message, puis, à la main, je vais mettre le nombre de tirets nécessaires.* »

Ce qui donnerait le code suivant :

```
tache: str = 'Faire les courses'
message: str = 'Tâche :'
#tache_message contiendra 'Tâche Faire les courses'
# la longueur compté à la main est de 24 caracteres
tache_message: str = message + tache
tache_message = tache_message + '\n-----------------------‘
print(tache_message)
```

Une autre approche, plus automatisée, peut être adoptée. Lorsque vous écrivez du code, il faut essayer de faire en sorte qu'il fonctionne peu importe le texte contenu. Dans notre cas, nous avons écrit les séparateurs à la main, donc, si je change le nom de la tâche, je dois à nouveau compter et modifier mes séparateurs.

Tous droits réservés à STUDI - Reproduction interdite

Ainsi, Python nous propose len(), une fonction native permettant de connaître la longueur d'une chaîne de caractères. Nous pouvons ensuite répéter un même symbole via l'opérateur '*'. Nous allons découper ce code en plusieurs étapes. La première étape sera de rajouter un séparateur de la taille de la chaîne de caractères.

```
1 tache: str = 'Faire les courses'
2 message: str = 'Tâche :'
3 #tache_message contiendra 'Tâche Faire les courses'
4 # la longueur compté à la main est de 24 caracteres
5 tache_message: str = message + tache
6 tache_message = tache_message + '\n' + ('-' * 24) print(tache_message)
```

Comme vous le voyez,ici, nous avons modifié la ligne pour ajouter 24 fois un séparateur '-'.

Maintenant, nous allons arrêter de mettre 24 en dur dans le code, mais, le prendre de manière dynamique via la fonction len()

```
1 tache: str = 'Faire les courses'
2 message: str = 'Tâche :'
3 #tache_message contiendra 'Tâche Faire les courses'
4 # la longueur compté à la main est de 24 caracteres
5 tache_message: str = message + tache
6 tache_message = tache_message + '\n' + ('-' * len(tache_message))
  print(tache_message)
```

José, s'est trompé dans le programme qu'il vous a donné !

Il ne voulait pas mettre '*Faire les courses*' mais '*Faire les achats*'. Il trouve cela plus clair car il doit acheter d'autres choses que des courses.

Pour modifier cela, python nous donne une fonction toute prête qui est « *replace* ». Elle est simple, elle a besoin de trois choses :

- Une chaine de caractères à analyser
- L'élément à trouver et à remplacer
- L'élément à mettre dans la chaine

Cela donne l'approche suivante :

```
1 tache: str = 'Faire les courses'
2 tache = tache.replace( 'courses', 'achats')
3 message: str = 'Tâche :'
4 #tache_message contiendra 'Tâche Faire les courses'
5 # la longueur £omoté à la main est de 24 sacasteces
6 tache_message: str = message + tache
7 tache_message = tache_message + '\n' + ('-' * len(tache_message))
8 print(tache_message)
```

Attention, replace est sensible à la casse. Cela veut dire que '*Courses*' n'est pas pareil que '*courses*'.

C. Accès et extraction de sous-chaînes

Méthode

Pour continuer notre exploration des chaînes de caractères, nous allons maintenant nous concentrer sur l'accès aux caractères d'une chaîne et leur extraction. En Python, nous appelons cela l'indexation et le slicing.

L'indexation est un élément fondamental des chaînes de caractères, permettant d'accéder à un caractère spécifique. Il est important de rappeler que Python, comme bon nombre de langages, commence à compter à partir de l'indice 0. Ainsi, le deuxième caractère est à l'indice 1, etc.

Tous droits réservés à STUDI - Reproduction interdite

Python permet également l'utilisation des indices négatifs, qui commencent à compter à partir de la fin de la chaîne, où -1 correspond au dernier caractère.

Supposons que nous ayons une chaîne « *Python* ». Nous pouvons accéder à chaque caractère en utilisant son indice.

```
1 chaine: str = "Python"
2 premier_caractere = chaine[0]
3 print(premier_caractere)
4 dernier caractere = chaine [-1]
5 print(dernier_caractere)
```

Dans cet exemple, chaine[0] retourne « *P* », qui est le premier caractère de la chaîne « *Python* ». De même, chaine[-1] retourne « *n* », le dernier caractère de la chaîne.

L'indexation permet d'obtenir les caractères de manière individuelle. Le slicing est complémentaire : il peut extraire plusieurs caractères en un seul coup !

Le slicing a besoin de quelques éléments pour fonctionner, notamment un indice de début et un indice de fin qui ne sont pas obligatoires.

Par exemple, nous pouvons demander à Python d'extraire les caractères de l'indice 1 à 5 (exclus) comme ceci :

```
1 chaine: str = "Python"
2 sous_chaine = chaine[1:5]
3 print(sous_chaine)
```

Allons encore loin ! Nous pouvons demander d'accéder aux caractères de deux en deux en débutant au caractère à l'indice 0, ou, sur un autre indice.

```
1 chaine: str = "Python"
2 # on va de deux en deux à partir de l'indice 0
3 sous_chaine_zero = chaine[::2]
4 print(sous_chaine_zero) # Pto
5 # on va de deux en deux à partir de l'indice 1
6 sous_chaine_un = chaine [1::2]
7 print(sous_chaine_un) # yhn
```

Revenons sur les indices négatifs, ils peuvent être très utiles pour un palindrome, par exemple « *noël* » et « *Léon* », ils sont des palindromes car Léon est l'inverse de noël.

Dans un premier temps, appréhendons les indices négatifs avec ce petit bout de code

```
1 chaine: str = "Python"
2 # allons de un en un, en arriere depuis le dernier caractère
3 sous_chaine = chaine [::-1]
4 print(sous_chaine) # nohtyP
```

Nous commençons à lire depuis le dernier caractère de la chaîne, mais nous allons en arrière au lieu d'aller en avant.

Dans cet exemple, nous avons commencé à la fin de la chaîne, mais nous pouvons très bien demander de commencer à l'indice 3, par exemple.

```
1 chaine: str = "Python"
2 # allons de un en un, en arriere depuis le dernier caractère
3 sous_chaine = chaine[3::-1]
4 print(sous_chaine) # htyP
```

Tentons d'adapter ce bout de code à notre palindrome. Dans un premier temps, nous allons avoir besoin d'un mot de base qui est « *Léon* » dans notre exemple, et d'un autre mot qui existe « *noël* », le but est de savoir si « *Léon* » à l'envers donne « *noël* »

```
1 chaine: str = "Leon"
2 chaine_palindrome: str = "noel"
3 # allons de un en un, en arriere depuis le dernier caractère
4 print(chaine[::-1] == chaine_palindrome) # True
```


Tous droits réservés à STUDI - Reproduction interdite

Cette écriture permet d'éviter les boucles et est beaucoup plus compréhensible et maintenable.

Le slicing permet d'extraire des segments non-continus, par exemple, nous avons un mot « *Pythonista* » et nous désirons les caractères des indices 0 à 2, puis, des indices 4 à 5.

```
1 chaine: str = "Pythonista"
2 # nous prenoms les caractères de l'index 0 à 3 exclus
3 segment_un_chaine = chaine[0:3] # Pyt
4 print(segment_un_chaine)
5 # je fais de même en commençant à l'index 4 jusque 6 exclus
6 segment_deux = chaine [3:6] # on
7 print(segment_deux)
8 # on effectue une concatenation afin d'avoir un resultat
9 resultat = segment_un_chaine + segment_deux
10 print(resultat) # Pyton
```

Le slicing, n'a pas forcément besoin d'indice de fin, si cela est absent, il va aller jusqu'au bout de la chaine, mais il va éliminer le premier caractère.

```
1 chaine: str = "Pythonista"
2 derniers_caracteres = chaine[1:]
3 print(derniers_caracteres) # ythonista
```

D. Exercice

Nous avons le code suivant :

```
1 chaine_un: str = "Bonjour"
2 chaine_deux: str = " le monde"
```

Question 1

[solution n°1 p. 28]

José souhaite vous tester et demande que vous effectuiez une concaténation et affichiez la longueur de la nouvelle chaîne via un print.

Question 2

[solution n°2 p. 28]

Ensuite, il vous demande de concevoir une chaine de caractères contenant « *Programmation* », vous devez :

- Accédez et afficher le troisième caractère ;
- Extraire et affichez les caractères de l'indice 4 à l'indice 8 ;
- Inversez et affichez la chaine.

Question 3

[solution n°3 p. 28]

Enfin, José mentionne que lorsqu'il se rend à la montagne, il crie « *écho* » et entend 3 fois « *écho* ». Il vous demande de reproduire ce comportement en Python en répétant 3 fois « *écho* » dans une string.

E. Exercice : Quiz

[solution n°4 p. 28]

Question 1

Quel opérateur est utilisé pour concaténer deux chaînes de caractères en Python ?

*

&

+

Tous droits réservés à STUDI - Reproduction interdite

-

Question 2

Quelle méthode est utilisée pour obtenir la longueur d'une chaîne de caractères en Python ?

length()

size()

len()

count()

Question 3

Que retourne l'expression '*chaine[1 :4]*' si 'chaine = « *Python* »' ?

Pyt

yth

tho

ytho

Question 4

Que fait l'expression '*chaine[::-1]*' ?

Retourne une copie de la chaîne sans le premier caractère

Retourne une copie de la chaîne inversée

Retourne une sous-chaîne de l'indice 0 à 1

Retourne une copie de la chaîne en majuscules

Tous droits réservés à STUDI - Reproduction interdite

III Manipulation des chaînes

A. Introduction

Les transformations sont essentielles pour manipuler et formater du texte de manière efficace. Nous allons voir dans cette partie comment convertir, nettoyer et transformer les chaînes de caractères.

B. Transformation

Méthode

Les éléments les plus basiques sont les fonctions upper() et lower(). L'une peut mettre une chaîne de caractères en majuscules, et l'autre en minuscules. Cette approche est souvent utilisée pour faire des comparaisons, car « *Course* » est différent de « *course* ». Ainsi, en mettant tout en minuscules ou en majuscules, on s'assure du bon fonctionnement de notre algorithme.

```
1 chaine = "Bonjour tout le monde"
2 chaine_upper = chaine.upper()
3 print(chaine_upper) # BONJOUR TOUT LE MONDE
4 chaine_lower = chaine.lower()
5 print(chaine_lower) # bonjour tout le monde
```

Il peut également arriver que « *chaîne* » contienne un titre, par exemple. Généralement, nous mettons des majuscules au début de chaque mot, ce qui peut être assez complexe à développer. Cependant, encore une fois, Python nous facilite la vie grâce à la méthode title().

```
1 chaine = "Bonjour tout le monde"
2 chaine_upper = chaine.upper()
3 print(chaine_upper) # BONJOUR TOUT LE MONDE
4 chaine_lower = chaine.lower()
5 print(chaine_lower) # bonjour tout le monde
6 chaine_lower = chaine.title()
7 print(chaine_lower) # Bonjour Tout Le Monde
```

Un autre cas de figure pourrait être de se dire : « *Je veux une majuscule juste au début de ma chaîne de caractères, mais pour le reste, je veux des minuscules.* » Pour ce faire, la méthode capitalize() existe. Cette fonction est très pratique car elle permet de manipuler des phrases ou des titres de manière standardisée (car dans la langue française, nous commençons nos phrases par une majuscule, et le reste est en minuscule).

```
1 chaine: str = "programmation"
2 chaine_capitalize = chaine.capitalize()
3 print(chaine_capitalize) # Programmation
```

À l'inverse, la méthode swapcase() permet de mettre le premier caractère de chaque mot en minuscule, et le reste en majuscule. Elle est moins utilisée, mais reste tout de même intéressante selon les besoins auxquels vous devez répondre.

```
1 chaine: str = "programmation python"
2 chaine_swapcase = chaine.swapcase()
3 print(chaine_swapcase) # pROGRAMMATION pYTHON
```

Une fonction très utile est casefold(). Son fonctionnement est similaire à celui de lower(), mais elle gère les éléments multilingues. Dans certains alphabets, la lettre '*ß*' est utilisée et signifie '*ss*'. Cette fonction est bien plus agressive dans la conversion des caractères en minuscules. Elle est surtout utilisée pour les comparaisons insensibles à la casse ou encore pour des applications de traduction de langues.

Tous droits réservés à STUDI - Reproduction interdite

```
1 mot_a_convertir = "straße"
2 mot_en_francais = "STRASSE"
3 if mot_a_convertir.casefold() == mot_en_francais.casefold():
4 print("Les noms sont équivalents.")
5 else:
6 print("Les noms sont différents.")
```

Vous pouvez essayer avec lower(), mais vous verrez que les noms sont différents.

Exemple

Maintenant que vous avez acquis des compétences sur les transformations de chaînes, José, l'apprenant qui vous a demandé de l'aide, vous sollicite à nouveau !

Il travaille sur une application de gestion des utilisateurs et doit effectuer plusieurs tâches pour s'assurer que les données sont normalisées et affichées correctement.

José reçoit des noms d'utilisateurs, mais il ne sait pas à l'avance comment ils sont constitués. Par exemple, il peut avoir « *ALICE* », « *alice* » ou encore « *aLiCE* ».

L'idée est de mettre le nom d'utilisateur en majuscule, afin de gagner en simplicité d'exploitation.

```
1 nom_utilisateur: str = "aliCE"
2 nom_utilisateur = nom_utilisateur.upper()
```

Ensuite, il vous explique que les utilisateurs possèdent un titre de profil qui est tout simplement le nom du poste. Par exemple, Alice a le poste de « *développeuse python senior* », et il souhaite que chaque mot commence par une majuscule.

```
1 nom_utilisateur: str = "aliCE"
2 nom_utilisateur = nom_utilisateur.upper()
3 Poste: str = "developpeuse python senior"
4 postę = poste.title()
```

Pour éviter les doublons de pseudonymes, José doit comparer les pseudonymes des utilisateurs sans tenir compte de la casse. Une première approche serait de se dire : « *Je vais tout mettre en minuscule* ».

```
1 pseudo_un = "Jean2024"
2 pseudo_deux = "jean2024"
3 # je met tout en minuscule
4 pseudo_un = pseudo_un.Lower()
5 pseudo_deux = pseudo_deux.lower()
6 if pseudo_un == pseudo_deux:
7 print("Pseudo identique")
8 else:
9 print("Pseudo different")
```

Le souci, c'est que José ne nous a pas dit s'il a une gestion multilingue, ainsi, notre code fonctionne, mais si on change les valeurs de pseudo_un et pseudo_deux, cela peut potentiellement ne plus fonctionner.

Nous allons utiliser casefold qui est plus sécurisée.

```
1 pseudo_un = "Jean2024"
2 pseudo_deux = "jean2024"
3 # je met tout en minuscule
4 pseudo_un = pseudo_un.casefold()
5 pseudo_deux = pseudo_deux.casefold()
6 if pseudo_un == pseudo_deux:
7 print("Pseudo identique")
8 else:
9 print("Pseudo different")
```


Tous droits réservés à STUDI - Reproduction interdite

C. Recherche et remplacement

Méthode

Python nous fournit plusieurs méthodes pour rechercher des sous-chaînes que nous allons aborder. La notion de recherche est importante pour l'analyse de texte ainsi que pour le nettoyage et le remplacement de contenu.

Les méthodes les plus utilisées sont *find()* et *replace()*.

La première va chercher un motif dans la chaîne de caractères et renvoyer son indice, tandis que la seconde permet d'effectuer un remplacement.

```
1 texte: str = "Bonjour tout le monde."
2 indice = texte.find("Bonjour")
3 print(indice) #0
```

Cependant, find() renvoie l'indice de la première occurrence trouvée. Par exemple, si nous avons deux fois « bonjour », le résultat sera toujours 0 pour la première occurrence. En revanche, si aucune occurrence n'existe, cela renvoie -1.

```
1 texte: str = "Bonjour tout le monde. Bonjour à tous."
2 indice = texte.find("Bonjour")
3 print(indice) #0
```

Nous pouvons aller plus loin en précisant un indice de début de recherche avec find(). Si nous voulons trouver le deuxième « *Bonjour* », nous pouvons écrire : texte.find("*Bonjour*", 2). Cela commence à chercher le mot « *Bonjour* » à partir de l'indice 2.

Si nous souhaitons trouver la dernière occurrence de « *Bonjour* » sans spécifier cet indice, nous pouvons utiliser rfind().

```
1 texte: str = "Bonjour tout le monde. Bonjour à tous."
2 indice = texte.rfind("Bonjour")
3 print(indice) #23
```

Si nous souhaitons obtenir les indices des deux occurrences de « *Bonjour* » (puisque le texte contient deux fois « *Bonjour* »), find() ne permet pas de le faire. Nous devons importer dans notre code re, un module Python qui gère les expressions régulières (plus communément appelées regex).

```
1 import re
2 texte: str = "Bonjour tout le monde. Bonjour à tous."
3 indice = re.findall(pattern: "Bonjour", texte)
4 print(indice) # ['Bonjour', 'Bonjour']
```

Le module re sera abordé un peu plus tard dans la partie "*Techniques avancées*" car il permet de faire énormément de choses. Mais avant cela, nous avons encore besoin d'acquérir certaines compétences.

Enfin, pour remplacer des chaînes, admettons qu'au lieu de dire « *Bonjour* » nous souhaitions utiliser « *Salut* ». Dans ce cas, nous devons indiquer à Python la chaîne de caractères que nous voulons modifier, l'ancien contenu, puis le nouveau contenu.

```
1 texte: str = "Bonjour tout le monde. Bonjour à tous. "
2 nouveau_texte: str = texte.replace("Bonjour","Salut")
3 print(nouveau_texte) # Salut tout le monde. Salut à tous.
```

Afin de vérifier si « *Bonjour* » est encore dans nouveau_texte, nous pouvons utiliser l'opérateur in. Cela permet de savoir si un mot est contenu dans une chaîne de caractères, et nous pouvons également l'utiliser avec des listes. Cela renvoie une valeur : soit vrai, soit faux.

Tous droits réservés à STUDI - Reproduction interdite

```
1 texte: str = "Bonjour tout le monde. Bonjour à tous."
2 nouveau_texte: str = texte.replace("Bonjour","Salut")
3 print(nouveau_texte) # Salut tout le monde. Salut à tous.
4 print('Bonjour' in nouveau_texte) # False
```

D. Séparation et jointure

Les chaînes de caractères en Python peuvent être manipulées de diverses manières pour répondre à des besoins spécifiques. La séparation et la jointure sont deux opérations très fréquemment utilisées, et elles sont à la fois pratiques et puissantes. On peut, à partir d'une chaîne de caractères, en obtenir plusieurs, et, à partir de plusieurs chaînes de caractères, en obtenir une seule.

Méthode

Pour ce faire, la méthode split() de Python est utilisée pour diviser une chaîne en une liste de sous-chaînes en utilisant un séparateur spécifié. Par défaut, le séparateur est un espace blanc, mais il peut être n'importe quel caractère ou ensemble de caractères.

```
1 texte: str = "Python est amusant"
2 mots = texte.split()
3 print(mots) # ['Python', 'est', 'amusant']
```

split() ne renvoie pas vraiment plusieurs chaines de caractères, mais un tableau contenant chaque élément séparé. On peut également préciser un séparateur.

```
1 texte: str = "Python, Java, C++, JavaScript"
2 langages = texte.split(',')
3 print(langages) # ['Python', 'Java', 'C++', 'JavaScript']
```

Nous n'avons pas encore tout vu ! split() peut également accepter un deuxième argument qui limite le nombre de divisions effectuées.

```
1 texte: str = "un; deux; trois; quatre"
2 limite = texte.split(';', 2)
3 print(limite) # ['un', 'deux', 'trois; quatre']
```

Il est important de rappeler que si on demande deux splits, cela renvoie trois valeurs car on compte à partir de 0. Si on veut vraiment une liste contenant seulement deux éléments, il faut spécifier un maximum de splits de 1.

À l'inverse, la méthode join() est utilisée pour joindre les éléments d'une liste en une seule chaîne en utilisant un délimiteur spécifié.

Si on reprend le premier exemple que l'on a effectué avec split(), faire l'inverse n'est pas très compliqué. La méthode join() a besoin de deux choses : le caractère de séparation à inclure, puis la liste des chaînes de caractères.

```
1 mots = ['Python', 'est', 'amusant']
2 phrase: str =' '.join(mots)
3 print (phrase) # Python est amusant
```

On peut également aller un peu plus loin en proposant avec un séparateur ',' , le changement n'est pas volumineux, nous devons juste changer le séparateur à la seconde ligne. Cela permet d'afficher le séparateur ',' entre chaque mot à concaténer.

```
1 langages = ['Python', 'Java', 'C++', 'JavaScript']
2 liste_langages: str = ', '.join(langages)
3 print(liste_langages) # Python, Java, C++, JavaScript
```


Tous droits réservés à STUDI - Reproduction interdite

Fondamental

Un split() permet de séparer une chaine de caractère en un tableau comportant plusieurs chaines de caractères. Le join() permet de faire l'inverse, il prend en argument un tableau de caractères et il a besoin d'un séparateur à appliquer afin de transformer le tableau de chaine de caractère en une seule chaine.

Méthode

Entraînons-nous un peu maintenant. José reçoit une liste d'adresses électroniques séparées par des points-virgules et doit les convertir en une liste de chaînes individuelles, puis les reformater en une seule chaîne séparée par des virgules.

Voici le programme actuel de José :

```
emails: str = alice@example.com; bob@example.com; charlie@example.com
```

José souhaite convertir cette liste en une liste de chaînes individuelles. Nous savons que nous devons utiliser split(). De plus, dans la chaîne de caractères, nous voyons que le séparateur est un « *;* ».

Ensuite, nous devons les joindre à nouveau pour obtenir une chaîne de caractères avec une « , » comme séparateur. Voici le code correspondant :

```
emails: str = "alice@example.com;bob@example.com;charlie@example.com"
# Séparation des adresses email
emails_separes = emails.split(';')
print(emails_separes) # ['alice@example.com', 'bob@example.com',
'charlie@example.com']
# Jointure des adresses email avec des virgules
emails_formates = ', '.join(emails_separes)
print(emails_formates) # alice@example.com, bob@example.com, charlie@example.com
```

Une approche plus simple et plus optimisée, serait d'utiliser replace(), une méthode que nous avons vu, et lui demander de remplacer tous les « *;* » en « , »

```
emails: str = "alice@example.com;bob@example.com;charlie@example.com"
# modification du separateur ";" par ","
# alice@example.com,bob@example.com,charlie@example.com
print(emails.replace(';',','))
```

Très souvent, il existe plusieurs solutions pour un même besoin. Le replace est très utile et permet de simplifier considérablement votre code.

Ensuite, José doit séparer les noms complets en prénoms et noms de famille, puis les joindre pour les afficher en majuscules avec un trait d'union entre eux.

```
noms_complets: str = "Alice Dupont, Bob Martin, Charlie Durand"
```

Dans un premier temps, nous allons effectuer un split avec le séparateur « , »

```
noms_complets: str = "Alice Dupont, Bob Martin, Charlie Durand"
noms = noms complets.split(', ')
```

« *noms* » est une liste, à ce titre pour accéder à « *Alice Dupont* » qui est le premier du tableau dans noms, nous allons devoir faire « *noms[0]* ». Nous prenons donc la chaine de caractères stockée à l'indice 0 de noms.

Tous droits réservés à STUDI - Reproduction interdite

```
noms complets: str = "Alice Dupont, Bob Martin, Charlie Durand"
noms = noms_complets.split(', ') # ['Alice Dupont', 'Bob Martin', 'Charlie Durand']
# je vais prendre les noms complet à un et les stocker.
nom_un = noms[0]
nom_deux = noms[1]
nom_trois = noms [2]
```

Au sein des noms, nous avons encore un caractère que nous souhaitons séparer : l'espace.

Pour ce faire, nous allons faire un replace.

```
noms complets: str = "Alice Dupont, Bob Martin, Charlie Durand"
noms = noms_complets.split(', ') # ['Alice Dupont', 'Bob Martin', 'Charlie Durand']
# je vais prendre les noms complet à un et les stocker.
nom_un = noms[0].replace(‘ ‘, ‘-‘)
nom_deux = noms[1].replace(‘ ‘, ‘-‘)
nom_trois = noms [2].replace(‘ ‘, ‘-‘)
```

Il ne nous reste plus qu'à effectuer une concaténation afin de passer de trois chaînes de caractères à une seule.

```
noms complets: str = "Alice Dupont, Bob Martin, Charlie Durand"
noms = noms_complets.split(', ') # ['Alice Dupont', 'Bob Martin', 'Charlie Durand']
# je vais prendre les noms complet à un et les stocker.
nom_un = noms[0].replace(‘ ‘, ‘-‘)
nom_deux = noms[1].replace(‘ ‘, ‘-‘)
nom_trois = noms [2].replace(‘ ‘, ‘-‘)
noms_formatter: str = nom_un + ‘\n’ + nom_deux + ‘\n’ + nom_trois
print(noms_formatter)
```

Nous obtenons le résultat prévu, c'est-à-dire :

```
Alice-Dupont
Bob-Martin
Charlie-Durand
```

Figure 56 : résultat de l'exécution du code

Cependant, nous pouvons simplifier la ligne de noms_formatter, en incluant un join. Join, est capable de prendre une string ou un tableau de string (ce qui donne en finalité, le même résultat que ce que nous venons de faire).

```
noms_formatter: str = '\n'.join([nom_un, nom_deux, nom_trois])
```


Tous droits réservés à STUDI - Reproduction interdite

E. Exercice : Quiz

[solution n°5 p. 30]

Question 1

Quelle est la sortie de ce code ?

```
1 texte: str = "Python, Java, C++, JavaScript"
2 resultat = texte.split(',')
3 print(resultat)
```

`['Python']`

`['Python,Java,C++,JavaScript']`

`['Python', 'Java', 'C++', 'JavaScript']`

`['Python', 'Java']`

Question 2

Quelle est la sortie de ce code ?

```
1 langages = ['Python', 'Java', 'C++', 'JavaScript']
2 resultat = ', '.join(langages)
3 print(resultat)
```

`Python Java C++ JavaScript`

`Python, Java, C++, JavaScript`

`['Python', 'Java', 'C++', 'JavaScript']`

`Python,Java,C++,JavaScript`

Question 3

Que fait la méthode `*replace()*` ?

Elle remplace toutes les occurrences d'une sous-chaîne par une autre sous-chaîne

Elle divise la chaîne en une liste de sous-chaînes

Elle retourne l'indice de la première occurrence d'une sous-chaîne

Elle enlève les espaces au début et à la fin d'une chaîne

Question 4

Que retourne la méthode `*find()*` si la sous-chaîne n'est pas trouvée ?

Tous droits réservés à STUDI - Reproduction interdite

0

-1

None

Une exception

Question 5

Quelle est la sortie de ce code ?

```
texte: str = "Python est amusant. Apprendre Python est intéressant."
position = texte.find("Python",10)
print(position)
```

`-1`

`0`

`30`

`35`

Question 6

Quelle est la sortie de ce code ?

```
texte: str = "Python est amusant. Apprendre Python est intéressant • "
position = texte.find("Python",10,30)
print(position)
```

`-1`

`0`

`24`

`35`

Question 7

Quelle est la sortie de ce code ?

```
texte = "Python est amusant."
position = texte.find("Java")
print(position)
```

`-1`

Tous droits réservés à STUDI - Reproduction interdite

`0`

`15`

`Python est amusant`

Question 8

Quelle est la sortie de ce code ?

```
1 texte: str = "Python"
2 position = texte.find("")
3 print(position)
```

`-1`

`0`

`6`

`IndexError`

Tous droits réservés à STUDI - Reproduction interdite

IV Techniques avancées

A. Introduction

Les techniques avancées de manipulation de chaînes de caractères permettent d'effectuer des opérations complexes de manière efficace et élégante. Dans cette partie, nous aborderons trois aspects principaux : le formatage des chaînes en Python, les expressions régulières, et les méthodes utiles pour le contrôle de séquence ainsi que la vérification des entrées et des données, tout en évoquant les nouveautés introduites dans Python 3.12.

B. Formatage

Méthode

Les f-strings, introduites dans Python 3.6, offrent une méthode concise et lisible pour incorporer des expressions dans les chaînes de caractères. Elles sont définies en préfixant la chaîne de caractères avec un f ou F, et les expressions sont placées entre des accolades {}.

En d'autres termes, au lieu de concaténer, ce qui entraîne une perte de performance, on peut directement injecter une variable dans une chaîne de caractères. La variable peut être de type numérique ou string.

Cela améliore la lisibilité tout en offrant un gain de performance.

```
1 nom: str = "Alice"
2 age: int = 30
3 message = f"Bonjour, je m'appelle {nom} et j'ai {age} ans."
4  print(message) # Bonjour, je m'appelle Alice et j'ai 30 ans.
```

Le « *f* » positionné devant le contenu de la chaine de caractères est une simplification d'écriture.

L'écriture complète la suivante, mais la première écriture est plus appréciée.

```
1 nom: str = "Alice"
2 age: int = 30
3 # Dans la premiere accolade, j'injecte le nom, puis dans l'autre, l'age
4 message = "Bonjour, je m'appelle {} et j'ai {} ans.".format(nom, age)
5 print(message) # Bonjour, je m'appelle Alice et j'ai 30 ans.
```

Python 3.12 introduit plusieurs améliorations aux f-strings, notamment la réutilisation des guillemets dans les f-strings (PEP 701) et le support des expressions multi-lignes. La PEP est un guide indiquant toutes les modifications des versions de Python et expose les meilleures pratiques du langage.

Avec cette amélioration, il est désormais possible de réutiliser les guillemets à l'intérieur des f-strings sans provoquer d'erreur de syntaxe.

Dans la première version que nous avions vue ensemble, nous avions utilisé des doubles guillemets, et non un mélange de simples et de doubles guillemets.

```
1 nom = "Alice"
2 message = f"Son nom est '{nom}'"
3  print(message) # Son nom est 'Alice'
```

Les f-strings et la méthode format() permettent de formater les nombres avec des spécifications précises, comme le nombre de décimales.

Prenons par exemple π (pi), dont la valeur n'est pas fixe. Avec les f-strings, on peut facilement afficher seulement deux décimales.

Tous droits réservés à STUDI - Reproduction interdite

```python
pi: float = 3.141592653589793
# Utilisation des f-strings
print(f"Pi avec 2 décimales: (pi:.2f}") # Pi axes 2 désinales: 3.14
# Utilisation de la méthode format()
print("Pi avec 2 décimales: {:.2f}".format(pi)) # Pi axes 2 décimales: 3.14
```

De même, cela peut s'effectuer sur des alignements d'une chaine de caractères pour apporter de l'esthétique. C'est un outil extrêmement performant et apportant une souplesse non négligeable.

```python
nom: str = "Alice"
age: int=30
# Utilisation des f-strings
print(f"{nom:<10} | {age:>5}") # Alice     |   30
# Utilisation de la méthode format()
print("{:<10} | {:>5}".format(nom, age)) # Alice     |   30
```

- `f"{nom:<10}"` : Cette expression formate la variable `nom` en lui réservant un espace de 10 caractères, aligné à gauche (`<`). Si `nom` a moins de 10 caractères, les espaces supplémentaires seront ajoutés à droite.
- `f"{age:>5}"` : Cette expression formate la variable `age` en lui réservant un espace de 5 caractères, aligné à droite (`>`). Si `age` a moins de 5 caractères, les espaces supplémentaires seront ajoutés à gauche.
- - Le résultat est `Alice     |   30`, où `Alice` est suivi de plusieurs espaces pour atteindre 10 caractères, et `30` est précédé de plusieurs espaces pour atteindre 5 caractères.
- `"{:<10}». format(nom)` : Cette expression formate la variable `nom` en lui réservant un espace de 10 caractères, aligné à gauche (`<`).
- `"{:>5}». format(age)` : Cette expression formate la variable `age` en lui réservant un espace de 5 caractères, aligné à droite (`>`).
- Le résultat est le même : `Alice     |   30`.

C. Expression régulières

Une expression régulière (regex) est une séquence de caractères qui forme un modèle de recherche. Ce modèle peut être utilisé pour rechercher, modifier ou manipuler des chaînes de caractères. Les regex sont des outils puissants pour l'analyse de texte et la validation de données. Bien qu'elles puissent sembler complexes au premier abord, elles offrent une flexibilité et une efficacité inégalées pour traiter des chaînes de caractères.

Méthode

Python fournit un module intégré appelé re pour travailler avec les expressions régulières. Nous avons déjà exploré un peu ce module, mais allons maintenant plus loin.

Dans un premier temps, la notion de recherche est simplifiée avec re. Nous avons search(), qui permet d'obtenir un résultat différent de None si quelque chose a été trouvé ; sinon, nous obtenons None en retour.

```python
import re
texte: str = "Bonjour, je m'appelle Alice."
resultat = re.search(r"Alice", texte)
if resultat:
  print("Correspondance trouvée!")
print(resultat.group()) # Alice
else:
print("Aucune correspondance trouvée.")
```

Tous droits réservés à STUDI - Reproduction interdite

Si, lorsque vous voulez afficher le résultat, vous n'utilisez pas .group(), le résultat affiché dans la console ne sera pas exploitable.

Dans un autre cas, nous pouvons avoir besoin d'obtenir toutes les occurrences du modèle dans une chaîne et de vouloir obtenir une liste de toutes les correspondances. En complément de la recherche, le module re nous permet d'ignorer les majuscules et minuscules via l'argument facultatif IGNORECASE. À ce titre, findall() est une méthode intéressante, car elle renvoie un tableau comportant tous les éléments qui correspondent avec l'expression.

```
1 import re
2 texte: str = "Chat, chien, cheval, chat."
3 correspondances = re.findall(r"chat", texte, re.IGNORECASE)
4 print(correspondances) # ['Chat', 'chat']
```

Ce module permet également de modifier toutes les occurrences d'un mot donné dans une chaîne de caractères par une autre, c'est une sorte de « *super replace* ».

```
1 import re
2 texte = "Bonjour, je m'appelle Alice, Alice m'a été donné par mes parents."
3 nouveau_texte = re.sub(r"Alice", "Bob", texte)
4 print(nouveau_texte) # Bonjour, je m'appelle Bob.
```

Enfin, ce module, sert aussi à faire du split. Il fonctionne de la même manière que le split classique :

```
1 import re
2 texte = "Bonjour; je m'appelle ; Alice."
3 morceaux = re.split(r";", texte)
4 print (morceaux) # ['Bonjour', je m'appelle', Alice.']
```

D. Fonction et méthodes utiles

Méthode

Une première méthode intéressante est strip(), qui permet de supprimer par défaut les espaces des deux côtés d'une chaîne. Cependant, nous pouvons vouloir supprimer les espaces uniquement à gauche ou à droite de la chaîne de caractères. Pour ce faire, rstrip() permet de retirer les espaces à droite, et lstrip() permet de retirer les espaces à gauche.

```
1 chaine: str = "   Bonjour tout le monde.   "
2 chaine_strip = chaine.strip()
3 chaine_lstrip = chaine.lstrip()
4 chaine_rstrip = chaine.rstrip()
5 print(f" '{chaine_strip}'") # 'Bonjour tout le monde'
6 print(f" '{chaine_lstrip}") # 'Bonjour tout le monde.  '
7 print(f" '{chaine_rstrip}'") # "   Bonjour tout le monde.   "
```

Après avoir formaté la chaîne de caractères avec strip(), on peut avoir besoin d'une autre méthode qui nous indique si, oui ou non, une chaîne de caractères commence ou se termine par une certaine sous-chaîne.

Python nous facilite la vie car nous pouvons utiliser startswith() ainsi que endswith(), qui répondent parfaitement à cette question !

```
1 chaine: str = "Bonjour tout le monde"
2 commence_par = chaine.startswith("Bonjour")
3 finit_par = chaine.endswith("monde")
4 print(commence_par) # True
5 print(finit_par) # True
```


Tous droits réservés à STUDI - Reproduction interdite

La fonction count(), permet de retourner le nombre de fois qu'une sous-chaîne apparaît dans une chaîne. Par exemple, si nous voulons savoir combien de fois nous avons le mot « *Bonjour* » dans une chaine de caractères, nous pouvons effectuer la chose suivante.

```
1 chaine = "Bonjour tout le monde, Bonjour tout le monde"
2 compte = chaine.count("Bonjour")
3 print (compte) #2
```

count() est une fonction qui peut être comparée à len(), que nous avons déjà vue. len() est une fonction qui peut être utilisée sur des chaînes de caractères, ainsi que sur des tableaux, tandis que count() est utilisé uniquement sur des chaînes de caractères. len() est généralement à préférer.

À l'inverse, index() renvoie l'indice de la première occurrence trouvée, si elle existe. Sinon, cela retourne une erreur de type ValueError. Une erreur provoque un arrêt du code, et si d'autres instructions devaient être exécutées, elles ne le seraient pas.

```
1 chaine: str = "Bonjour tout le monde"
2 indice = chaine.index("tout")
3 print(indice) #8
```

Python offre des méthodes pour vérifier le contenu des chaînes de caractères, ce qui est particulièrement utile et fondamental pour la validation des entrées utilisateur. Faire confiance à un utilisateur est souvent une faille de sécurité, en tant que développeur, nous nous devons de toujours contrôler nos données.

Pour ce faire, les cinq fonctions fondamentales : `isalnum()`, `isalpha()`, `isdigit()`, `islower()`, `isupper()`

Ces méthodes permettent de vérifier si les caractères d'une chaîne répondent à certains critères :

- `isalnum()` : Vérifie si tous les caractères sont alphanumériques.
- `isalpha()` : Vérifie si tous les caractères sont des lettres.
- `isdigit()` : Vérifie si tous les caractères sont des chiffres.
- `islower()` : Vérifie si tous les caractères sont en minuscules.
- `isupper()` : Vérifie si tous les caractères sont en majuscules.

```
1 chaine_un = "abc123"
2 chaine_deux = "ABC"
3 chaine_trois = "123"
4 chaine_quatre = "abc"
5 chaine_cinq = "ABC123"
6 print(chaine_un.isalnum()) # True
7 print(chaine_deux.isalpha()) # True
8 print(chaine_trois.isdigit()) # True
9 print(chaine_quatre.islower()) # True
10 print(chaine_cinq.isupper()) # True
```

E. Exercice : Quiz

[solution n°6 p. 34]

Question 1

Quelle est la sortie de ce code utilisant des f-strings ?

```
1 nom: str = "Alice"
2 age: int = 30
3 message: str = f"Bonjour, je m'appelle {nom} et j'ai {age} ans."
4 print(message)
```

`Bonjour, je m'appelle Alice et j'ai 30 ans.`

Tous droits réservés à STUDI - Reproduction interdite

`Bonjour, je m'appelle {nom} et j'ai {age} ans.`

`Bonjour, je m'appelle ALICE et j'ai 30 ans.`

`Bonjour, je m'appelle ALICE et j'ai {age} ans.`

Question 2

Quelle est la sortie de ce code ?

```
1 pi: float = 3.141592653589793
2 message: str = "Pi arrondi à deux décimales : {pi:.2f}"
3 print(message)
```

`Pi arrondi à deux décimales : 3.1416`

`Pi arrondi à deux décimales : 3.14`

`Pi arrondi à deux décimales : 3.1`

`Pi arrondi à deux décimales : 3.141592653589793`

Question 3

Quelle est la sortie de ce code ?

```
1 a = 5
2 b = 10
3 message=f"La somme de {a} et {b} est {a + b}"
4 print (message)
```

`La somme de 5 et 10 est 15`

`La somme de {a} et {b} est {a + b}`

`La somme de 5 et 10 est {a + b}`

`La somme de 5 et 10 est a + b`

Question 4

Quelle est la sortie de ce code ?

```
1 nom: str = "Alice"
2 message: str = f"{nom: <10} est ici."
3 print (message)
```

`Alice      est ici.`

Tous droits réservés à STUDI - Reproduction interdite

`Alice est ici.`

`    Alice est ici.`

`Alice----- est ici.`

Question 5

Quelle est la sortie de ce code utilisant la méthode `format()` ?

```
1 nom: str = "Alice"
2 age: int = 30
3 message = "Bonjour, je m'appelle {} et j'ai {} ans.".format(nom, age)
4 print(message)
```

`Bonjour, je m'appelle Alice et j'ai 30 ans.`

`Bonjour, je m'appelle {nom} et j'ai {age} ans.`

`Bonjour, je m'appelle ALICE et j'ai 30 ans.`

`Bonjour, je m'appelle ALICE et j'ai {age} ans.`

Question 6

Quelle est la sortie de ce code ?

```
1 nom: str = "Alice"
2 age: int = 30
3 message = "{:<10} | {:>5}".format(nom, age)
4 print (message)
```

`Alice     |    30`

`Alice|30`

`Alice     | 30`

`Alice|    30`

Question 7

Quelle est la sortie de ce code ?

```
1 chaine: str = " Bonjour tout le monde."
2 resultat = chaine.strip()
3 print("{resultat}")
```

`' Bonjour tout le monde '`

Tous droits réservés à STUDI - Reproduction interdite

`'Bonjour tout le monde'`

`' Bonjour tout le monde '`

`'Bonjour tout le monde '`

Tous droits réservés à STUDI - Reproduction interdite

V Essentiel

Les chaînes de caractères en Python sont des séquences immuables de caractères utilisées pour stocker et manipuler du texte. La création de chaînes peut se faire avec des guillemets simples, doubles, ou triples pour les chaînes multi-lignes. Les opérations de base incluent la concaténation (assemblage de chaînes), la répétition, et l'accès à des sous-chaînes par indexation et slicing. L'usage de len() permet de mesurer la longueur d'une chaîne, tandis que des méthodes comme split() et join() sont cruciales pour séparer et combiner des chaînes, respectivement.

Pour transformer les chaînes, Python offre des méthodes comme upper(), lower(), et capitalize() qui modifient la casse des caractères. Les opérations de recherche et de remplacement sont facilitées par find(), rfind(), et replace(). La séparation des chaînes avec split() et leur combinaison avec join() sont essentielles pour manipuler du texte, tout comme les fonctions de nettoyage des espaces superflus avec strip(), lstrip(), et rstrip().

La validation des entrées utilisateur est une autre utilisation importante des chaînes. Les méthodes startswith() et endswith() vérifient les préfixes et suffixes d'une chaîne, tandis que isalnum(), isalpha(), et isdigit() vérifient le contenu des chaînes pour s'assurer qu'elles contiennent respectivement des caractères alphanumériques, alphabétiques, ou numériques.

Les f-strings, introduites en Python 3.6, ont révolutionné le formatage des chaînes. Elles permettent d'incorporer des expressions dans des chaînes de manière concise et lisible. Par exemple, f"Bonjour, {nom}" insère directement la valeur de la variable nom. Les f-strings permettent aussi des formats numériques avancés et l'alignement du texte. Python 3.12 a ajouté des fonctionnalités pour les f-strings, incluant la possibilité d'utiliser des expressions multi-lignes.

Les expressions régulières (regex) sont des outils puissants pour la recherche et la manipulation de texte. En utilisant le module re, vous pouvez rechercher des motifs complexes, valider des formats (comme les adresses électroniques), et effectuer des remplacements sophistiqués dans des chaînes. Des fonctions comme re.search(), re.findall(), et re.sub() facilitent ces opérations.

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

[exercice p. 9]

Solution n°1

```
chaine_un: str = "Bonjour"
chaine deux: str = " le monde"
concatenation = chaine_un + chaine_deux
print(concatenation) # Bonjour le monde
print(len(concatenation)) # 15
```

[exercice p. 9]

Solution n°2

```
chaine "Programmation"
#j'accède au troisième caractère et je le stock
troisieme_caractere = chaine [2]
print(troisieme_caractere) # o
# je prends les caractères de l'indice 4 à 9 exclus
sous_chaine = chaine [4:9]
print(sous_chaine) # ramma
#j'inverse la chaine et je la stock
chaine_inversee = chaine [::-1]
print(chaine_inversee) # noitammargorP
```

[exercice p. 9]

Solution n°3

```
mot : str = « Echo »
repetition = mot * 3
Print(repetition)
```

Solution n°4

[exercice p. 9]

Question 1

Quel opérateur est utilisé pour concaténer deux chaînes de caractères en Python ?

- *
- &
- ✓ +
- -

En Python, l'opérateur + est utilisé pour concaténer deux chaînes de caractères. Cela signifie qu'il combine deux chaînes en une seule.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Quelle méthode est utilisée pour obtenir la longueur d'une chaîne de caractères en Python ?

- length()
- size()
- ✓ len()
- count()

En Python, la méthode len() est utilisée pour obtenir la longueur d'une chaîne de caractères, c'est-à-dire le nombre de caractères qu'elle contient. Cette fonction peut également être utilisée pour d'autres types de séquences, comme les listes ou les tuples, pour en déterminer la taille.

Question 3

Que retourne l'expression '*chaine[1 :4]*' si 'chaine = « *Python* »' ?

- Pyt
- ✓ yth
- tho
- ytho

En Python, l'expression chaine[1:4] est utilisée pour obtenir une sous-chaîne, appelée "*slice*", de la chaîne. Voici comment cela fonctionne :

- L'indice 1 correspond au deuxième caractère de la chaîne (car l'indexation commence à 0 en Python).
- L'indice 4 est exclusif, c'est-à-dire que la sous-chaîne prendra les caractères jusqu'à l'indice 3 inclus.

Donc, si chaine = "*Python*", l'expression chaine[1:4] extrait les caractères situés aux indices 1, 2, et 3, soit "*yth*".

Question 4

Que fait l'expression '*chaine[::-1]*' ?

- Retourne une copie de la chaîne sans le premier caractère
- ✓ Retourne une copie de la chaîne inversée

Tous droits réservés à STUDI - Reproduction interdite

Retourne une sous-chaîne de l'indice 0 à 1

Retourne une copie de la chaîne en majuscules

L'expression chaine[::-1] est une forme de slicing en Python qui permet d'inverser une chaîne de caractères. Voici comment cela fonctionne :

- L'absence de valeurs avant les deux points signifie que le slicing commence au début de la chaîne.
- Le -1 à la fin indique que le slicing se fait en parcourant la chaîne de droite à gauche (c'est-à-dire à l'envers).

Solution n°5

[exercice p. 17]

Question 1

Quelle est la sortie de ce code ?

```
1 texte: str = "Python, Java, C++, JavaScript"
2 resultat = texte.split(',')
3 print(resultat)
```

`['Python']`

`['Python,Java,C++,JavaScript']`

✓ `['Python', 'Java', 'C++', 'JavaScript']`

`['Python', 'Java']`

Dans ce code, la méthode split(',') est utilisée sur la chaîne de caractères "*Python,Java,C++,JavaScript*". La méthode split() divise la chaîne en une liste en utilisant la virgule , comme délimiteur.

Voici ce qui se passe :

- La chaîne "*Python,Java,C++,JavaScript*" est divisée à chaque virgule.
- Le résultat est une liste contenant chaque partie de la chaîne séparée par des virgules.

  Ainsi, le print(resultat) affichera ['Python', '*Java*', '*C++*', '*JavaScript*'].

Question 2

Quelle est la sortie de ce code ?

```
1 langages = ['Python', 'Java', 'C++', 'JavaScript']
2 resultat = ', '.join(langages)
3 print(resultat)
```


Tous droits réservés à STUDI - Reproduction interdite

`Python Java C++ JavaScript`

✓ `Python, Java, C++, JavaScript`

`['Python', 'Java', 'C++', 'JavaScript']`

`Python,Java,C++,JavaScript`

Dans ce code, la méthode join() est utilisée pour concaténer les éléments de la liste langages en une seule chaîne de caractères, en insérant une virgule suivie d'un espace (,) entre chaque élément de la liste.

Voici ce qui se passe :

- La liste langages = ['*Python*', '*Java*', '*C++*', '*JavaScript*'] contient quatre chaînes de caractères.
- La méthode ', '.join(langages) crée une nouvelle chaîne de caractères en insérant ', ' entre chaque élément de la liste.
- Le résultat est donc la chaîne "*Python, Java, C++, JavaScript*".

Question 3

Que fait la méthode `*replace()*` ?

✓ Elle remplace toutes les occurrences d'une sous-chaîne par une autre sous-chaîne

Elle divise la chaîne en une liste de sous-chaînes

Elle retourne l'indice de la première occurrence d'une sous-chaîne

Elle enlève les espaces au début et à la fin d'une chaîne

La méthode replace() en Python est utilisée pour remplacer toutes les occurrences d'une sous-chaîne donnée par une autre sous-chaîne dans la chaîne d'origine.

Question 4

Que retourne la méthode `*find()*` si la sous-chaîne n'est pas trouvée ?

0

✓ -1

None

Tous droits réservés à STUDI - Reproduction interdite

Une exception

La méthode find() en Python recherche la première occurrence d'une sous-chaîne dans une chaîne. Si la sous-chaîne est trouvée, find() retourne l'index de sa première occurrence. Si la sous-chaîne n'est pas trouvée, find() retourne -1.

Question 5

Quelle est la sortie de ce code ?

```
1 texte: str = "Python est amusant. Apprendre Python est intéressant."
2 position = texte.find("Python",10)
3 print(position)
```

`-1`

`0`

✓ `30`

`35`

La méthode find("*Python*", 10) est utilisée pour rechercher la première occurrence de la sous-chaîne "*Python*" dans la chaîne texte, en commençant la recherche à partir de l'index 10.

Voici les étapes détaillées :

1. La chaîne texte est : "*Python est amusant. Apprendre Python est intéressant.*"
2. Le code texte.find("*Python*", 10) indique à Python de commencer la recherche de "*Python*" à partir de l'index 10.
3. La première occurrence de "*Python*" après l'index 10 se trouve à l'index 30 dans la chaîne.

Donc, print(position) affichera 30.

Question 6

Quelle est la sortie de ce code ?

```
1 texte: str = "Python est amusant. Apprendre Python est intéressant • "
2 position = texte.find("Python",10,30)
3 print(position)
```

✓ `-1`

`0`

`24`

Tous droits réservés à STUDI - Reproduction interdite

`35`

La méthode find("*Python*", 10, 30) est utilisée pour rechercher la sous-chaîne "*Python*" dans la chaîne texte, mais uniquement dans la plage d'index allant de 10 à 29 (l'index final 30 est exclusif).

Voici les étapes détaillées :

1. La chaîne texte est : "*Python est amusant. Apprendre Python est intéressant.*"
2. Le code texte.find("*Python*", 10, 30) indique à Python de chercher "Python" en commençant à l'index 10 et en terminant à l'index 29.
3. Dans la plage spécifiée (indices 10 à 29), la sous-chaîne "*Python*" n'apparaît pas. En effet, la prochaine occurrence de "Python" est à l'index 24, mais elle se termine à l'index 30, ce qui est en dehors de la plage spécifiée pour find.

Question 7

Quelle est la sortie de ce code ?

```
1 texte = "Python est amusant."
2 position = texte.find("Java")
3 print(position)
```

✓ `-1`

`0`

`15`

`Python est amusant`

La méthode find("*Java*") est utilisée pour rechercher la sous-chaîne "*Java*" dans la chaîne texte, qui est "*Python est amusant.*".

Voici ce qui se passe :

- La méthode find() recherche la sous-chaîne "*Java*" dans la chaîne texte.
- Comme "*Java*" n'est pas présent dans la chaîne "*Python est amusant.*", la méthode find() ne trouve rien et retourne -1.

Question 8

Quelle est la sortie de ce code ?

```
1 texte: str = "Python"
2 position = texte.find("")
3 print(position)
```

`-1`

Tous droits réservés à STUDI - Reproduction interdite

✓ `0`

`6`

`IndexError`

La méthode find("") est utilisée pour rechercher une chaîne vide ("") dans la chaîne texte, qui est "*Python*".

Voici ce qui se passe :

- En Python, lorsqu'on utilise la méthode find() avec une chaîne vide, elle retourne l'index du premier caractère de la chaîne, ce qui est toujours 0.
- Cela signifie que même si la chaîne que vous cherchez est vide, find("") retourne 0 pour indiquer le début de la chaîne.

Solution n°6

[exercice p. 23]

Question 1

Quelle est la sortie de ce code utilisant des f-strings ?

```
1 nom: str = "Alice"
2 age: int = 30
3 message: str = f"Bonjour, je m'appelle {nom} et j'ai {age} ans."
4 print(message)
```

✓ `Bonjour, je m'appelle Alice et j'ai 30 ans.`

`Bonjour, je m'appelle {nom} et j'ai {age} ans.`

`Bonjour, je m'appelle ALICE et j'ai 30 ans.`

`Bonjour, je m'appelle ALICE et j'ai {age} ans.`

Le code utilise une f-string (f pour « *formatted string* ») en Python, qui permet d'intégrer des expressions Python directement dans une chaîne de caractères en les encadrant avec des accolades {}.

Voici comment cela fonctionne :

- La variable nom contient la valeur « *Alice* ».
- La variable age contient la valeur 30.
- Dans la f-string, {nom} est remplacé par « *Alice* » et {age} est remplacé par 30.

Question 2

Quelle est la sortie de ce code ?

Tous droits réservés à STUDI - Reproduction interdite

```
1 pi: float = 3.141592653589793
2 message: str = "Pi arrondi à deux décimales : {pi:.2f}"
3 print(message)
```

`Pi arrondi à deux décimales : 3.1416`

✓ `Pi arrondi à deux décimales : 3.14`

`Pi arrondi à deux décimales : 3.1`

`Pi arrondi à deux décimales : 3.141592653589793`

Une f-string est utilisée pour formater un nombre flottant (pi) en une chaîne de caractères. L'expression {pi:.2f} dans la f-string a une signification particulière :

- pi fait référence à la variable pi, qui est définie comme 3.141592653589793.
- .2f indique que le nombre doit être formaté en un nombre à virgule flottante avec exactement 2 chiffres après la virgule.

  Ainsi, le nombre 3.141592653589793 est arrondi à 3.14 lorsqu'il est formaté avec .2f.

Question 3

Quelle est la sortie de ce code ?

```
1 a = 5
2 b = 10
3 message=f"La somme de {a} et {b} est {a + b}"
4 print (message)
```

✓ `La somme de 5 et 10 est 15`

`La somme de {a} et {b} est {a + b}`

`La somme de 5 et 10 est {a + b}`

`La somme de 5 et 10 est a + b`

Une f-string est utilisée pour formater une chaîne de caractères qui inclut les valeurs des variables a et b, ainsi que le résultat de leur somme (a + b).

Voici ce qui se passe :

- {a} est remplacé par la valeur de a, qui est 5.
- {b} est remplacé par la valeur de b, qui est 10.
- {a + b} est une expression Python qui calcule la somme de a et b, donc 5 + 10, ce qui donne 15.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quelle est la sortie de ce code ?

```
nom: str = "Alice"
message: str = f"{nom: <10} est ici."
print (message)
```

✓ `Alice     est ici.`

`Alice est ici.`

`     Alice est ici.`

`Alice----- est ici.`

Une f-string est utilisée avec un spécificateur de format {nom:<10}. Voici ce que cela signifie :

- 10 indique que la chaîne doit être alignée à gauche (<) dans un espace de 10 caractères de large.
- La chaîne « *Alice* » contient 5 caractères.
- En utilisant {nom:<10}, Python ajoute 5 espaces après « *Alice* » pour que la largeur totale soit de 10 caractères.

Question 5

Quelle est la sortie de ce code utilisant la méthode `format()` ?

```
nom: str = "Alice"
age: int = 30
message = "Bonjour, je m'appelle {} et j'ai {} ans.".format(nom, age)
print(message)
```

✓ `Bonjour, je m'appelle Alice et j'ai 30 ans.`

`Bonjour, je m'appelle {nom} et j'ai {age} ans.`

`Bonjour, je m'appelle ALICE et j'ai 30 ans.`

`Bonjour, je m'appelle ALICE et j'ai {age} ans.`

Tous droits réservés à STUDI - Reproduction interdite

Le code utilise la méthode format() pour insérer les valeurs des variables nom et age dans la chaîne de caractères. Voici comment cela fonctionne :

- {} dans la chaîne de caractères est un espace réservé qui sera remplacé par les arguments fournis à format().
- .format(nom, age) signifie que la première paire de crochets {} sera remplacée par la valeur de nom, et la deuxième paire de crochets {} sera remplacée par la valeur de age.

Question 6

Quelle est la sortie de ce code ?

```
nom: str = "Alice"
age: int = 30
message = "{:<10} | {:>5}".format(nom, age)
print (message)
```

✓ `Alice     |    30`

`Alice|30`

`Alice     | 30`

`Alice|    30`

Le code utilise la méthode format() avec des spécificateurs de format pour aligner le texte dans la chaîne de caractères. Voici ce que signifient les spécificateurs utilisés :

- {:<10} : Aligne le texte à gauche (<) dans un espace de 10 caractères. Ici, la valeur de nom ("Alice") est placée à gauche, et les espaces restants (5 espaces) remplissent les 10 caractères.
- {:>5} : Aligne le texte à droite (>) dans un espace de 5 caractères. Ici, la valeur de age (30) est placée à droite, avec 3 espaces devant pour remplir les 5 caractères.

Question 7

Quelle est la sortie de ce code ?

```
chaine: str = " Bonjour tout le monde."
resultat = chaine.strip()
print("{resultat}")
```

`' Bonjour tout le monde '`

✓ `'Bonjour tout le monde'`

`' Bonjour tout le monde '`

Tous droits réservés à STUDI - Reproduction interdite

`'Bonjour tout le monde '`

Dans ce code, la méthode strip() est utilisée pour enlever les espaces (ou d'autres caractères spécifiés) au début et à la fin de la chaîne de caractères.

Voici ce qui se passe :

- La chaîne chaine contient des espaces avant et après le texte "Bonjour tout le monde".
- La méthode chaine.strip() supprime tous les espaces en début et en fin de chaîne, mais ne touche pas aux espaces à l'intérieur de la chaîne.

  Ainsi, après l'application de strip(), la variable resultat contient "Bonjour tout le monde" sans les espaces supplémentaires au début et à la fin.

Tous droits réservés à STUDI - Reproduction interdite